

UNIVERSIDADE FEDERAL DE SANTA CATARINA

Elaboração: Valentina Ragnini

Tutorial de simulação VHDL no Quartus II

Adaptação do Tutorial 1 de Xilinx ISE e ISim para Quartus II e ModelSim Altera

Material tutorial do projeto de pesquisa

Plano de trabalho

2026

1 APRESENTAÇÃO

1.1 OBJETIVO E RESULTADO

Este roteiro reúne os três projetos do Tutorial 1: meio somador de 1 bit, somador de 4 bits e flip-flop D com três arquiteturas. A configuração das ferramentas é apresentada no primeiro projeto. Os seguintes detalham as mudanças nos circuitos e nos testbenches, com simulação no ModelSim-Altera.

O meio somador foi simulado por 50 ns. O somador de 4 bits recebeu um teste dirigido de 50 ns e as 256 combinações de entrada em 2560 ns. As três arquiteturas do flip-flop foram simuladas por 100 ns, com os mesmos estímulos. A conferência das saídas foi visual. A programação de uma placa FPGA não faz parte desta etapa.

1.2 AMBIENTE UTILIZADO

Quartus II 13.1.0 Build 162, Web Edition, 64 bits; ModelSim-Altera Starter Edition 10.1d; Windows. Os nomes dos menus e os prints correspondem a essas versões. Em outras versões, a organização das opções e o simulador disponível podem mudar.

1.3 ARQUIVOS NECESSÁRIOS

Projeto 1: Tut1_Proj1.vhd e Tut1_Proj1_TB.vhd.
Projeto 2: Tut1_Proj2.vhd, Tut1_Proj2_TB1.vhd e Tut1_Proj2_TB2.vhd.
Projeto 3: Tut1_Proj3_Mods.vhd, Tut1_Proj3_Top.vhd e Tut1_Proj3_TB1.vhd.
Os fontes foram extraídos do pacote Tut1_sources. No Projeto 3, ajustaram-se os estímulos e criaram-se as cópias TB2 e TB3 descritas na seção 12.

1.4 PREPARAÇÃO DAS PASTAS

Deve-se criar uma pasta com permissão de escrita, evitando espaços, acentos e caracteres especiais. Exemplo: C:\FPGA\tutoriais_quartus\projeto1_halfadd. Os dois arquivos VHDL devem ser copiados para essa pasta. Nos prints, o projeto está na pasta de documentos da autora; esse endereço deve ser adaptado ao computador utilizado.

O projeto pode ficar no disco interno mesmo quando Quartus e ModelSim estão instalados em um SSD externo. Nesse caso, o SSD deve permanecer conectado durante o uso. Para continuar em outro dia, será necessário abrir o arquivo halfadd.qpf.

2 DESCRIÇÃO DO CIRCUITO

Um meio somador soma dois bits, A e B. A saída Sum guarda o bit menos significativo do resultado, e Carry guarda o “vai um”. Quando A e B valem 1, a soma é 10 em binário: Sum vale 0 e Carry vale 1.

2.1 CÓDIGO DO CIRCUITO EM TUT1_PROJ1 VHD

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd is
  port (A, B : in std_logic;
        Sum, Carry : out std_logic);
end halfadd;

architecture halfadd of halfadd is
begin
  Sum <= A xor B;
  Carry <= A and B;
end halfadd;
```

A transcrição omite os comentários do fonte e preserva sua lógica. O slide original chama a arquitetura de comp; o arquivo fornecido e utilizado chama a arquitetura de halfadd. Ambas as denominações são válidas quando o encerramento usa o nome correspondente.

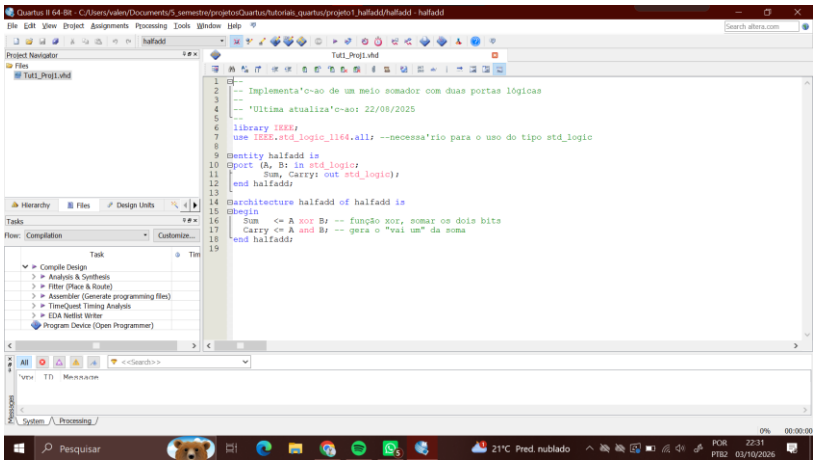
2.2 INTERPRETAÇÃO DO CÓDIGO

library IEEE e use IEEE.std_logic_1164.all disponibilizam o pacote que define std_logic e as operações lógicas. std_logic representa um sinal digital, incluindo os valores 0 e 1 e estados como valor desconhecido.

entity halfadd declara o módulo e suas portas. in indica entrada; out indica saída. Cada porta é de 1 bit. A arquitetura descreve o funcionamento do módulo.

O operador <= atribui um valor a um sinal. XOR produz 1 quando A e B são diferentes. AND produz 1 quando ambas as entradas valem 1. As duas atribuições são concorrentes: representam lógica que atua em paralelo, e não uma sequência de instruções de um programa comum.

Figura 1 — Arquivo do meio somador aberto no Quartus.



Fonte: captura de tela da autora (2026).

3 DESCRIÇÃO DO TESTBENCH

O testbench gera os valores das entradas e conecta o circuito a sinais internos. A entidade halfadd_tb é o módulo principal da simulação. Ela não possui portas externas porque os estímulos são produzidos dentro do teste.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd_tb is
end halfadd_tb;

architecture TB_ARCHITECTURE of halfadd_tb is
  signal aa, bb, soma, vaium : std_logic;
begin
  UUT : entity work.halfadd
    port map (A => aa, B => bb,
              Sum => soma, Carry => vaium);

  aa <= '0', '1' after 10 ns,
        '0' after 20 ns, '1' after 30 ns;
  bb <= '0', '1' after 20 ns;
end TB_ARCHITECTURE;
```

3.1 SINAIS E CONEXÕES

aa e bb são os fios virtuais ligados às entradas A e B; soma e vaium recebem as saídas Sum e Carry. Esses são os nomes que aparecerão nas ondas. Os nomes do testbench podem ser diferentes dos nomes das portas do circuito.

UUT identifica a instância do circuito sob teste, do inglês Unit Under Test. Instanciar é usar um módulo dentro de outro. work é a biblioteca de trabalho onde a entidade halfadd compilada fica disponível.

port map conecta as portas do meio somador aos sinais do teste. À esquerda de => aparece a porta do circuito; à direita, o sinal conectado. Assim, A => aa conecta A a aa; Carry => vaium conecta Carry a vaium.

3.2 TEMPOS DOS ESTÍMULOS

aa começa em 0, muda para 1 aos 10 ns, para 0 aos 20 ns e para 1 aos 30 ns. bb começa em 0 e muda para 1 aos 20 ns. Os tempos de after são contados a partir da execução dessas atribuições, no início da simulação; não são somados entre si. Após a última mudança, o valor permanece até o fim do teste.

Este testbench não contém comparações automáticas nem assert para aprovar as saídas. A validação é feita pela leitura das ondas e comparação com os resultados esperados. Seus estímulos com after são usados para simulação; o testbench não deve ser incluído como circuito a sintetizar.

4 CRIAÇÃO DO PROJETO NO QUARTUS

1. Abre-se o Quartus II e seleciona-se File → New Project Wizard. Caso seja exibida a introdução, deve-se clicar em Next.

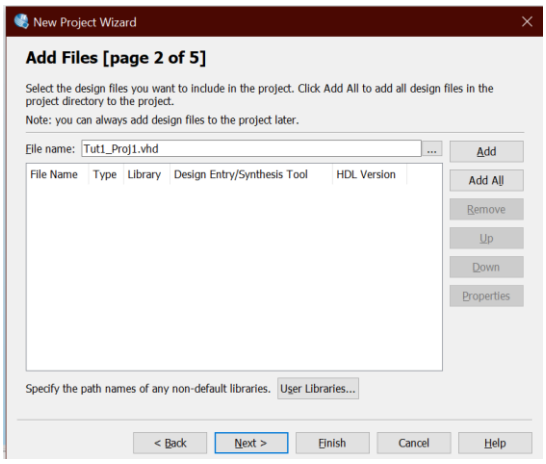
2. Em Working directory, informa-se a pasta preparada para o projeto. Em Project name e Top-level design entity, digita-se halfadd. Em seguida, deve-se clicar em Next.

3. Na etapa Add Files, deve-se clicar em … e selecionar Tut1_Proj1.vhd. Após clicar em Add, verifica-se se o arquivo aparece na lista. Em seguida, deve-se clicar em Next.

4. Adiciona-se somente o arquivo do circuito nesta etapa. O arquivo Tut1_Proj1_TB.vhd será cadastrado nas configurações de simulação.

O nome do arquivo e o nome da entidade são coisas distintas. O arquivo se chama Tut1_Proj1.vhd, mas declara entity halfadd. Por isso, o campo Top-level design entity deve conter halfadd.

Figura 2 — Seleção do fonte na etapa Add Files. Neste print ainda é necessário clicar em Add.



Fonte: captura de tela da autora (2026).

4.1 CONFERÊNCIA DOS NOMES

Para síntese, o módulo principal é halfadd. Para simulação, o módulo principal será halfadd_tb. Manter essa distinção evita tentar sintetizar o gerador de estímulos ou simular apenas o circuito sem entradas definidas.

Ao concluir o assistente, o Quartus cria os arquivos de projeto. O arquivo .qpf abre o projeto, e o arquivo .qsf armazena suas configurações. A pasta inteira deve ser preservada, junto com os fontes.

5 SELEÇÃO DO DISPOSITIVO E DO SIMULADOR

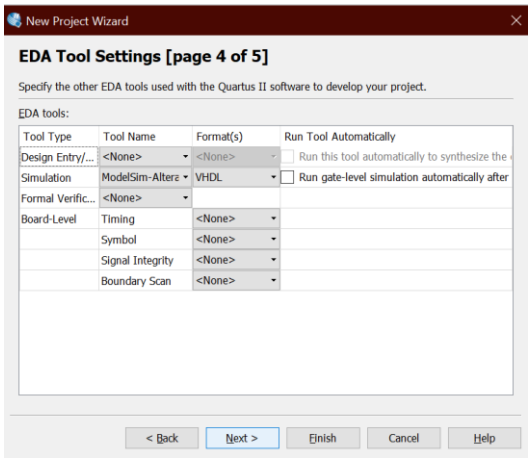
1. Na etapa Family & Device Settings, seleciona-se Cyclone III. Nesta execução foi usada a opção Auto device selected by the Fitter. Em seguida, deve-se clicar em Next.

2. Em EDA Tool Settings, na linha Simulation, selecionam-se ModelSim-Altera e o formato VHDL. Em seguida, deve-se clicar em Next.

3. Na tela Summary, verificam-se a pasta, o nome halfadd, a entidade principal halfadd e a inclusão de Tut1_Proj1.vhd. Após essa conferência, deve-se clicar em Finish.

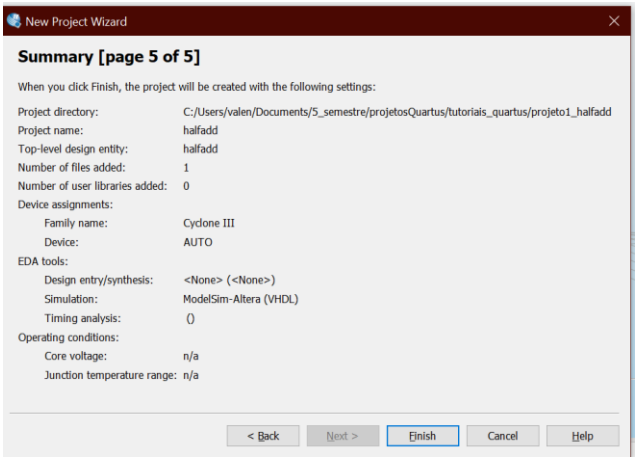
Cyclone III foi a família usada nesta atividade. Para simular a lógica simples deste meio somador não é necessário conectar uma placa. Quando o objetivo for implementar em hardware, a família, o dispositivo e os pinos deverão corresponder à placa real. Os dispositivos Spartan e a placa Nexys do roteiro Xilinx não devem ser copiados para um projeto Quartus.

Figura 3 — Seleção de ModelSim-Altera e VHDL no assistente.



Fonte: captura de tela da autora (2026).

Figura 4 — Resumo do projeto antes de finalizar o assistente.



Fonte: captura de tela da autora (2026).

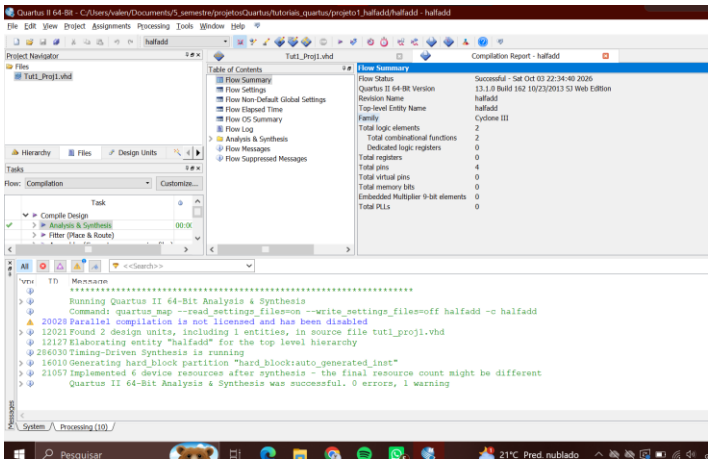
6 EXECUÇÃO DA ANÁLISE E SÍNTESE

1. Abre-se Tut1_Proj1.vhd pela aba Files do Project Navigator e verifica-se a declaração entity halfadd.

2. Seleciona-se Processing → Start → Start Analysis & Synthesis. Também é possível iniciar Analysis & Synthesis pelo painel Tasks.

3. Aguarda-se o fim do processamento e verificam-se as mensagens. A execução deve terminar sem erros. O resultado pode ser consultado no Compilation Report.

Figura 5 — Análise e síntese concluídas com sucesso.



Fonte: captura de tela da autora (2026).

Nesta execução foram registrados 2 elementos lógicos, 2 funções combinacionais, 0 registradores e 4 pinos. O resultado é compatível com o circuito: duas entradas, duas saídas e nenhuma memória. Os números de recursos podem variar com a família e a versão da ferramenta.

O aviso 20028, “Parallel compilation is not licensed and has been disabled”, informa que a compilação paralela não está habilitada pela licença utilizada. A execução mostrada terminou com 0 erros; esse aviso não impediu a análise e síntese.

6.1 O QUE FOI VERIFICADO NESTA ETAPA

A análise e síntese conferem e traduzem o código do circuito em uma representação de hardware. Essa conclusão ainda não demonstra que todas as combinações de entrada produzem as saídas corretas; essa verificação será feita no ModelSim.

Não é necessário executar a compilação completa com Fitter e geração de arquivo de programação para este teste RTL. O testbench será compilado pelo simulador quando a simulação for iniciada.

7 CONFIGURAÇÃO DO CAMINHO DO MODELSIM

1. Seleciona-se Tools → Options → EDA Tool Options.

2. Na linha ModelSim-Altera, informa-se a pasta que contém os executáveis do simulador. A linha ModelSim corresponde a outra opção de ferramenta e não deve ser usada nesta configuração.

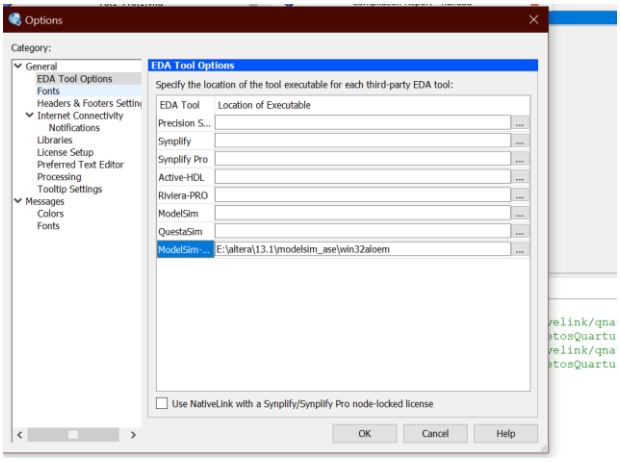
3. Deve-se clicar em OK para salvar a configuração.

No computador da execução, o caminho utilizado foi o seguinte. A barra final foi acrescentada para resolver o erro de localização ocorrido com estas versões:

```
E:\altera\13.1\modelsim_ase\win32aloem\
```

A letra da unidade e a pasta da instalação devem ser ajustadas conforme o computador. Informa-se a pasta, sem acrescentar vsim.exe e sem aspas. No Explorador, Ctrl + L seleciona a barra de endereço; Ctrl + C copia o endereço da pasta. A caixa de pesquisa do Explorador não serve para abrir um caminho diretamente.

Figura 6 — Campo ModelSim-Altera. O print foi obtido antes da correção; o endereço final deve incluir a barra após win32aloem.



Fonte: captura de tela da autora (2026).

7.1 VERIFICAÇÃO DA INSTALAÇÃO

Deve-se localizar vsim.exe na pasta indicada; quando o Windows oculta extensões, ele aparece como vsim. A abertura manual desse aplicativo permite conferir se a interface do ModelSim inicia. Essa instância de teste deve ser fechada antes de iniciar a simulação pelo Quartus.

A pasta também deve conter os programas de compilação, como vcom.exe e vlog.exe. A abertura manual da interface, sozinha, não garante que toda a integração com o Quartus esteja configurada.

8 CADASTRO DO TESTBENCH

1. Seleciona-se Assignments → Settings → EDA Tool Settings → Simulation. Verificam-se Tool name = ModelSim-Altera e Format for output netlist = VHDL. A opção Run gate-level simulation automatically after compilation deve permanecer desmarcada.

2. Em NativeLink settings, seleciona-se Compile test bench. Em seguida, deve-se clicar em Test Benches… e depois em New….

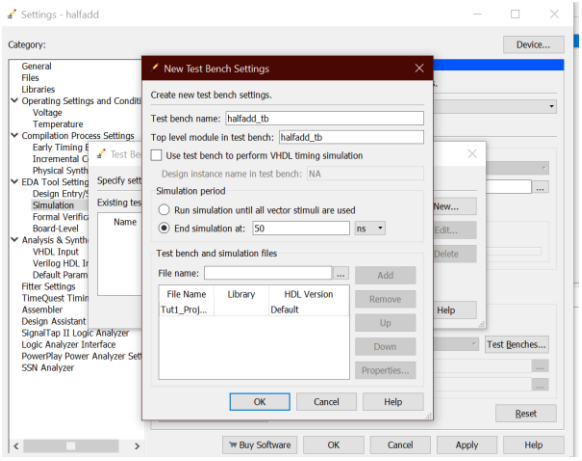
3. Em Test bench name e Top level module in test bench, digita-se halfadd_tb.

4. A opção Use test bench to perform VHDL timing simulation deve permanecer desmarcada. O campo Design instance name permanecerá desabilitado.

5. Em Simulation period, seleciona-se End simulation at, digita-se 50 e seleciona-se ns.

6. Em File name, deve-se clicar em …, selecionar Tut1_Proj1_TB.vhd e clicar em Add. Verifica-se a presença do arquivo na lista abaixo.

Figura 7 — Testbench cadastrado com duração de 50 ns e timing desmarcado.



Fonte: captura de tela da autora (2026).

Deve-se clicar em OK na janela New Test Bench Settings e novamente em OK em Test Benches. Na tela Simulation, verifica-se a seleção de halfadd_tb ao lado de Compile test bench. Em seguida, deve-se clicar em OK para salvar.

NativeLink é a integração que prepara os comandos e abre o simulador a partir do Quartus. Os 50 ns correspondem ao tempo virtual observado no circuito, e não ao tempo de espera no computador.

9 SIMULAÇÃO E INTERPRETAÇÃO DAS ONDAS

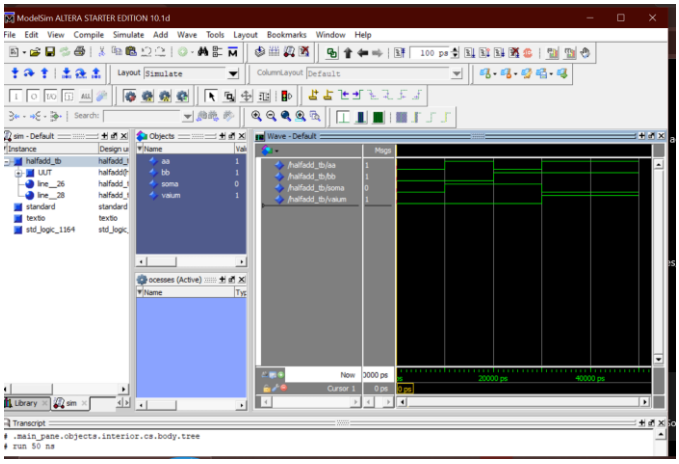
1. No Quartus, seleciona-se Tools → Run Simulation Tool → RTL Simulation. Aguarda-se a abertura do ModelSim e a compilação dos arquivos.

2. Verifica-se a presença de halfadd_tb e da instância UUT na hierarquia, além dos sinais aa, bb, soma e vaium na janela Wave.

3. Verifica-se se a simulação chegou a 50 ns. Se a unidade mostrada for ps, 50000 ps equivalem a 50 ns.

4. Deve-se clicar na área das ondas e pressionar F para usar Zoom Full. Também é possível usar a lupa com quatro setas azuis acima das ondas. O ajuste mostra os 50 ns completos; na abertura, a janela mostrava apenas uma pequena parte do tempo.

Figura 8 — Formas de onda do meio somador em 50 ns. As entradas e saídas correspondem aos quatro casos esperados.



Fonte: captura de tela da autora (2026).

Cada sinal tem seu próprio nível baixo, correspondente a 0, e alto, correspondente a 1. A leitura dos valores deve ser feita de cima para baixo em um mesmo instante. Os números ao lado dos nomes não substituem a leitura das ondas ao longo do tempo.

De 0 a 10 ns: aa = 0 e bb = 0; soma = 0 e vaium = 0.

De 10 a 20 ns: aa = 1 e bb = 0; soma = 1 e vaium = 0.

De 20 a 30 ns: aa = 0 e bb = 1; soma = 1 e vaium = 0.

De 30 a 50 ns: aa = 1 e bb = 1; soma = 0 e vaium = 1.

A simulação registrada reproduziu a tabela verdade completa do meio somador. As saídas confirmaram Sum = A XOR B e Carry = A AND B. A validação desta atividade foi visual, sem verificações automáticas no testbench.

10 SOLUÇÃO DE PROBLEMAS E RETOMADA DO PROJETO

10.1 QUARTUS NÃO ENCONTRA O MODELSIM

Caso apareça “Can’t launch the ModelSim-Altera software”, devem ser verificados Tools → Options → EDA Tool Options e a linha ModelSim-Altera. A pasta indicada deve existir e conter vsim.exe. Na execução registrada, incluir uma barra inversa no final do endereço resolveu esse erro. Trata-se de uma solução observada para o ambiente usado, não de uma garantia para todas as versões.

Caso o erro persista, deve-se consultar halfadd_nativelink_simulation.rpt, na pasta do projeto, com o Bloco de Notas. Esse registro descreve em qual etapa o NativeLink falhou. Também deve ser verificado o simulador selecionado em Assignments → Settings → EDA Tool Settings → Simulation.

10.2 MENSAGEM DE ENTIDADE NÃO ENCONTRADA

Caso a hierarquia mostre “Can’t find design entity halfadd”, deve-se verificar se Tut1_Proj1.vhd consta em Files, se o código declara entity halfadd e se a entidade principal do projeto é halfadd. Executa-se Analysis & Synthesis para atualizar a análise. Na sessão registrada, o fonte estava correto e a análise e síntese concluíram com sucesso.

10.3 MODELSIM APARENTA ESTAR TRAVADO OU AS ONDAS PARECEM CONSTANTES

Durante a abertura, o Windows pode indicar temporariamente “Não respondendo”. Deve-se aguardar a carga inicial e evitar iniciar outra instância. Caso a condição persista por alguns minutos, as mensagens devem ser verificadas antes do encerramento. O estado temporário da janela não deve ser tratado como prova de sucesso ou de falha do circuito.

Caso as ondas pareçam constantes, deve-se verificar o tempo final e usar Zoom Full. Neste teste, as mudanças ocorrem aos 10, 20 e 30 ns. Uma visualização de apenas 1 ns não permite observar as transições.

10.4 RETOMADA EM OUTRO DIA

Os programas devem ser fechados e as configurações salvas quando solicitado. A pasta do projeto e os dois fontes devem ser preservados. Antes da remoção do SSD externo, deve-se desligar o computador ou ejetar a unidade após o fechamento dos aplicativos. Na retomada, conecta-se o SSD, verifica-se sua letra de unidade e abre-se halfadd.qpf. Executa-se RTL Simulation novamente para reconstruir o teste e utiliza-se Zoom Full.

10.5 REGISTROS PARA UMA NOVA APLICAÇÃO EM AULA

Devem ser registradas a síntese sem erros, as configurações do testbench e a forma de onda com os quatro casos visíveis. Caso haja mudança de computador ou versão, os prints e o caminho da instalação devem ser atualizados. Este documento conserva os prints da execução realizada; não apresenta capturas reconstruídas.

11 PROJETO 2 SOMADOR DE 4 BITS

11.1 CIRCUITO E SÍNTESE

O arquivo Tut1_Proj2.vhd declara a entidade soma4b. As entradas A e B e a saída Sum são vetores de 4 bits, definidos por std_logic_vector(3 downto 0). Cada entrada representa um inteiro sem sinal de 0 a 15. A saída mantém os quatro bits menos significativos da soma, sem uma porta Carry.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity soma4b is
  port (A, B : in std_logic_vector(3 downto 0);
        Sum : out std_logic_vector(3 downto 0));
end soma4b;

architecture soma4b of soma4b is
begin
  Sum <= A + B;
end soma4b;
```

O código acima omite apenas os comentários do fonte. O pacote std_logic_unsigned, já presente no material fornecido, permite usar o operador + nos vetores como valores sem sinal. Como Sum possui 4 bits, o resultado corresponde à soma módulo 16: por exemplo, 9 + 8 = 17 produz 0001 na saída.

Criou-se um projeto separado, somador4, pelo procedimento da seção 4, incluindo somente Tut1_Proj2.vhd e definindo Top-level design entity como soma4b. Mantiveram-se Cyclone III, ModelSim-Altera e VHDL. O caminho do simulador já configurado foi reutilizado.

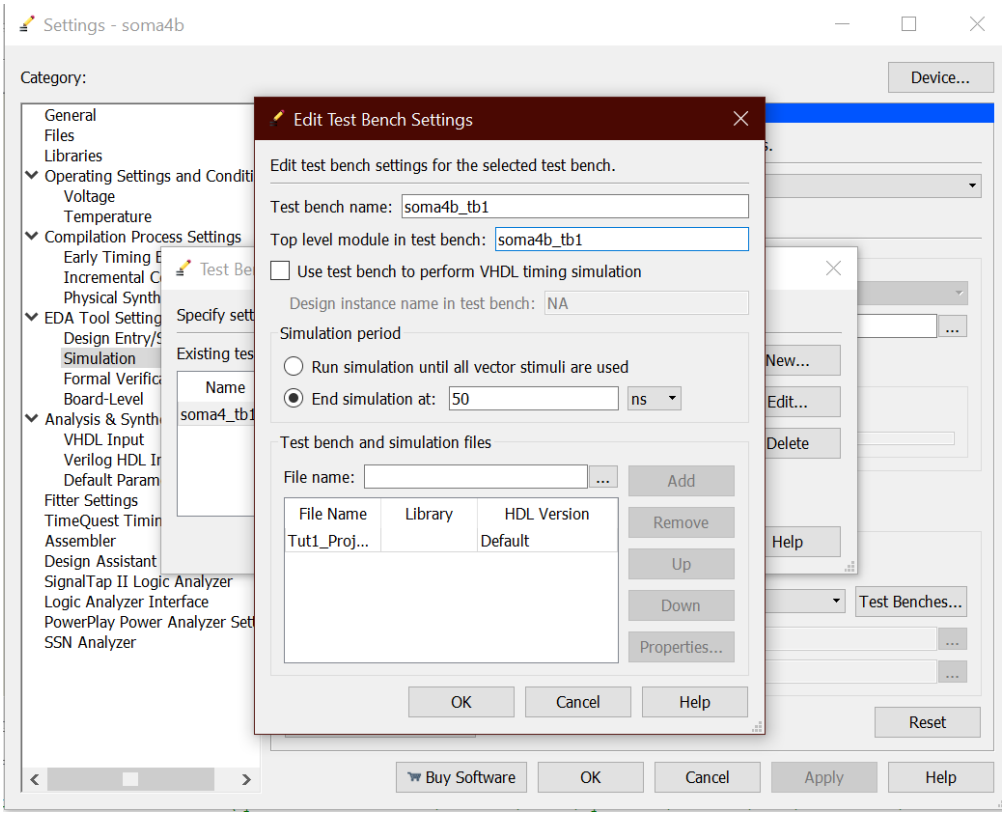
Executou-se Processing → Start → Start Analysis & Synthesis. O processamento terminou com 0 erros, 4 elementos lógicos, 4 funções combinacionais, 0 registradores e 12 pinos. Os pinos correspondem aos oito bits de entrada e aos quatro de saída. O aviso de licença de compilação paralela foi o mesmo explicado na seção 6.

11.2 PRIMEIRO TESTBENCH

O arquivo Tut1_Proj2_TB1.vhd instancia soma4b e conecta A a aa, B a bb e Sum a soma. Em hexadecimal, aa recebe 9, 5, A e B nos instantes 0, 10, 20 e 30 ns; bb começa em 8 e muda para 3 aos 20 ns.

Em Assignments → Settings → EDA Tool Settings → Simulation → Test Benches → New, cadastrou-se soma4b_tb1 nos campos Test bench name e Top level module in test bench. Selecionou-se End simulation at = 50 ns e adicionou-se Tut1_Proj2_TB1.vhd pelo campo File name e botão Add. A opção de VHDL timing simulation permaneceu desmarcada.

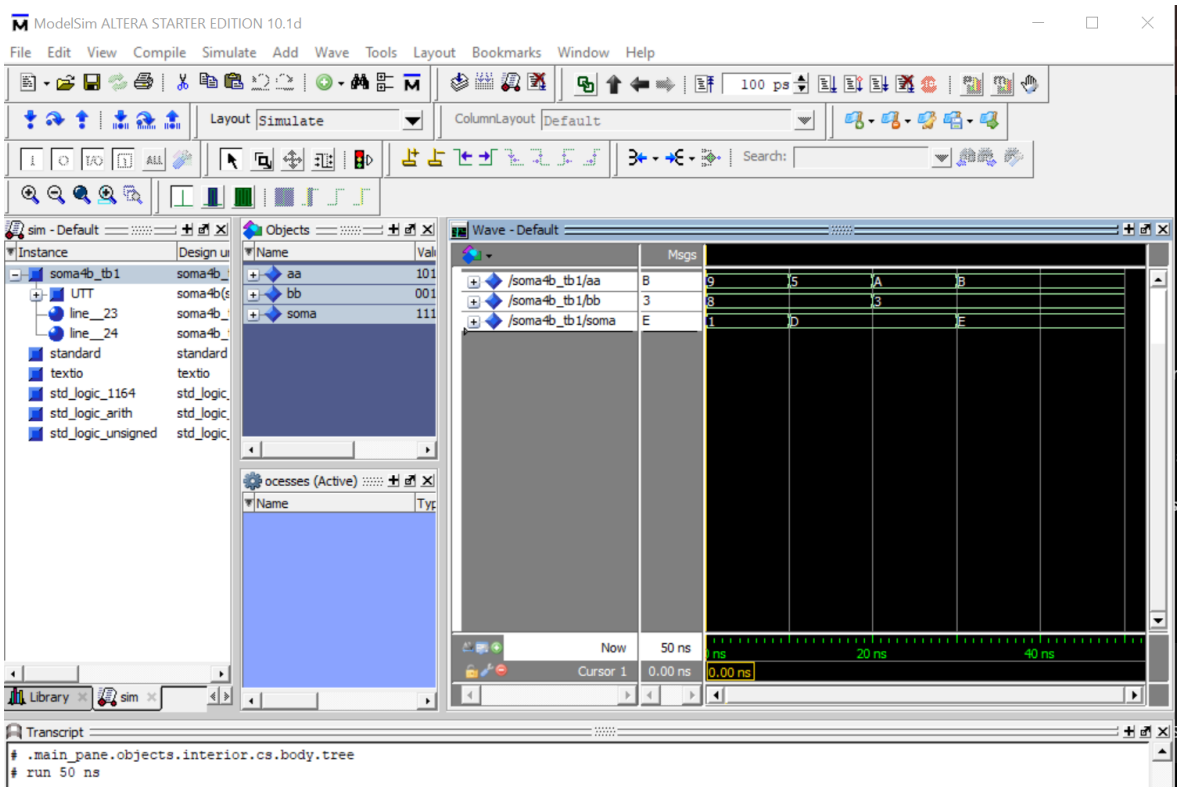
Figura 9 — Configuração de soma4b_tb1 para 50 ns.



Fonte: captura de tela da autora (2026).

Após confirmar as janelas, selecionou-se soma4b_tb1 em Compile test bench e iniciou-se Tools → Run Simulation Tool → RTL Simulation. Na janela Wave do ModelSim, selecionaram-se aa, bb e soma e aplicou-se Radix → Hexadecimal pelo menu de contexto. Zoom Full exibiu os 50 ns do teste.

Figura 10 — Resultados do primeiro testbench em hexadecimal.



Fonte: captura de tela da autora (2026).

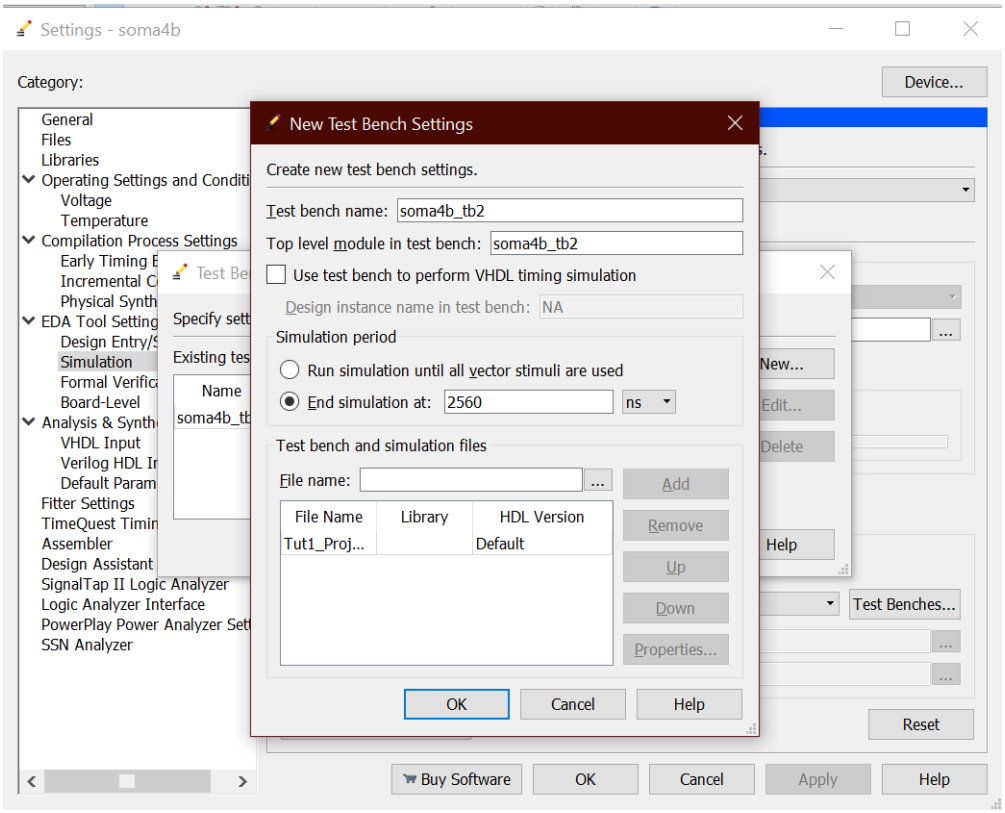
De 0 a 10 ns, 9 + 8 produziu 1; de 10 a 20 ns, 5 + 8 produziu D; de 20 a 30 ns, A + 3 produziu D; de 30 a 50 ns, B + 3 produziu E. Em hexadecimal, A, B, D e E representam 10, 11, 13 e 14. O primeiro caso evidencia o descarte do quinto bit da soma.

11.3 SEGUNDO TESTBENCH

O arquivo Tut1_Proj2_TB2.vhd percorre todas as combinações das entradas. Um processo incrementa aa a cada 10 ns e retorna a 0 após F. Outro incrementa bb a cada 160 ns. Assim, para cada valor de bb, aa percorre seus 16 valores. Um ciclo completo contém 16 × 16 = 256 combinações e dura 2560 ns. A conferência foi visual nos trechos apresentados, sem assert para verificar automaticamente todas as saídas.

Fechou-se o ModelSim e cadastrou-se outro teste em Test Benches → New, preservando o primeiro. Informou-se soma4b_tb2 nos dois campos de nome, adicionou-se Tut1_Proj2_TB2.vhd e definiu-se End simulation at = 2560 ns. As demais opções permaneceram iguais às do primeiro teste.

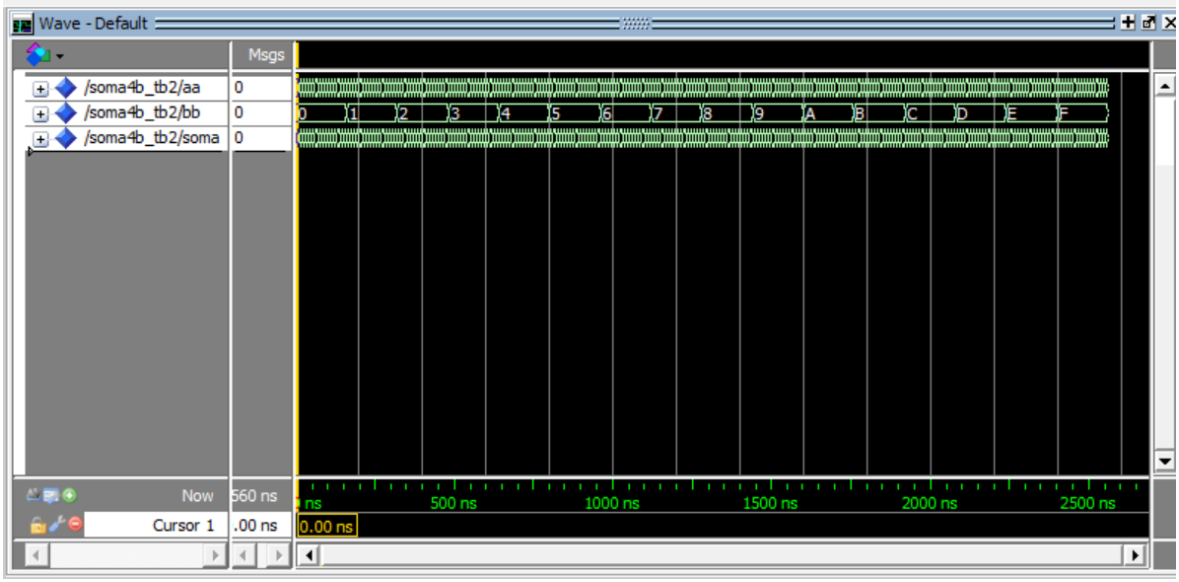
Figura 11 — Configuração de soma4b_tb2 para 2560 ns.



Fonte: captura de tela da autora (2026).

Selecionou-se soma4b_tb2 em Compile test bench e executou-se novamente RTL Simulation. Mantiveram-se os sinais em hexadecimal e utilizou-se Zoom Full para observar o ciclo completo. A entidade principal do projeto para síntese continuou sendo soma4b.

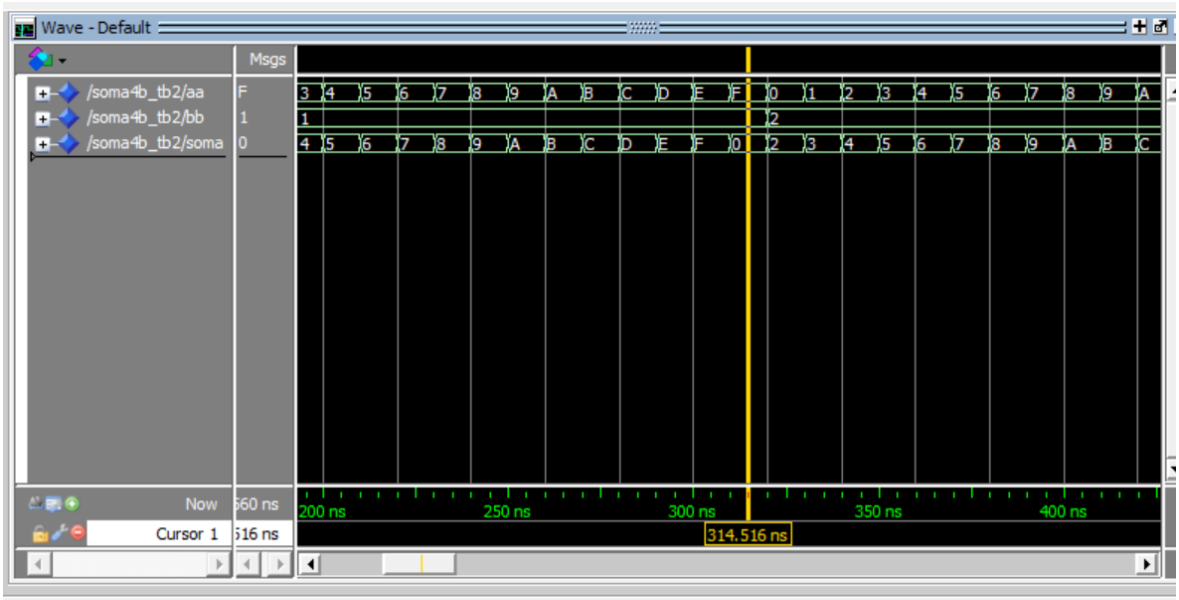
Figura 12 — Ciclo completo do segundo testbench em hexadecimal.



Fonte: captura de tela da autora (2026).

Na visão geral, bb percorre 0 a F, enquanto aa varia 16 vezes para cada valor de bb. Para ler os resultados individuais, ampliou-se a escala de tempo com Zoom In e posicionou-se o cursor em um intervalo entre transições.

Figura 13 — Resultado F + 1 observado aos 314,516 ns.



Fonte: captura de tela da autora (2026).

No instante destacado, aa = F e bb = 1, mas soma = 0. O resultado completo, 16, corresponde a 10000 em binário; a saída de 4 bits conserva 0000. Nos intervalos anteriores, a onda também mostra a sequência de resultados 4, 5, 6, …, F para bb = 1.

12 PROJETO 3 FLIP FLOP D

12.1 ARQUITETURAS E CONFIGURAÇÃO NO QUARTUS

A entidade flip_flop_d_es armazena d em q na borda de subida de clk. Os controles assíncronos pre_n e clr_n são ativos em 0: o primeiro força q = 1 e o segundo, q = 0. Não devem ser ativados simultaneamente.

Tut1_Proj3_Top.vhd contém codeonly_arch, com seis NAND descritas diretamente; nand3i_arch, com seis instâncias nand3i; e latch_arch, com três instâncias latch_4ctrls. Tut1_Proj3_Mods.vhd fornece os módulos hierárquicos.

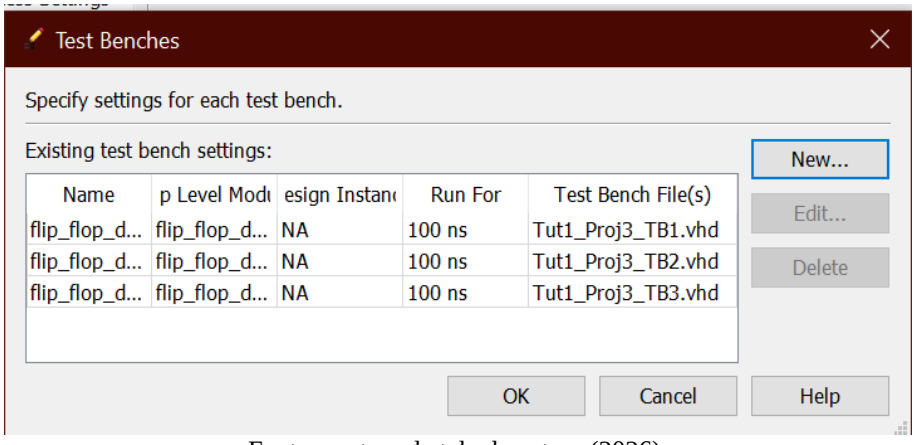
Criou-se o projeto flipflop, com Top-level design entity = flip_flop_d_es, adicionando os arquivos Mods e Top. Mantiveram-se Cyclone III, ModelSim-Altera e VHDL, conforme a configuração do primeiro projeto.

Copiou-se Tut1_Proj3_TB1.vhd para criar TB2 e TB3. Alteraram-se os nomes de entidade e arquitetura e seus encerramentos. No TB2, corrigiu-se o end que ainda indicava tb1. A instância UUT selecionou cada arquitetura:

```
TB1: entity work.flip_flop_d_es(codeonly_arch)
TB2: entity work.flip_flop_d_es(nand3i_arch)
TB3: entity work.flip_flop_d_es(latch_arch)
```

Em Assignments → Settings → EDA Tool Settings → Simulation → Test Benches, cadastraram-se flip_flop_d_es_tb1, flip_flop_d_es_tb2 e flip_flop_d_es_tb3, cada um com o arquivo correspondente. Definiu-se End simulation at = 100 ns e manteve-se VHDL timing simulation desmarcado.

Figura 14 — Três testbenches cadastrados para 100 ns.



Fonte: captura de tela da autora (2026).

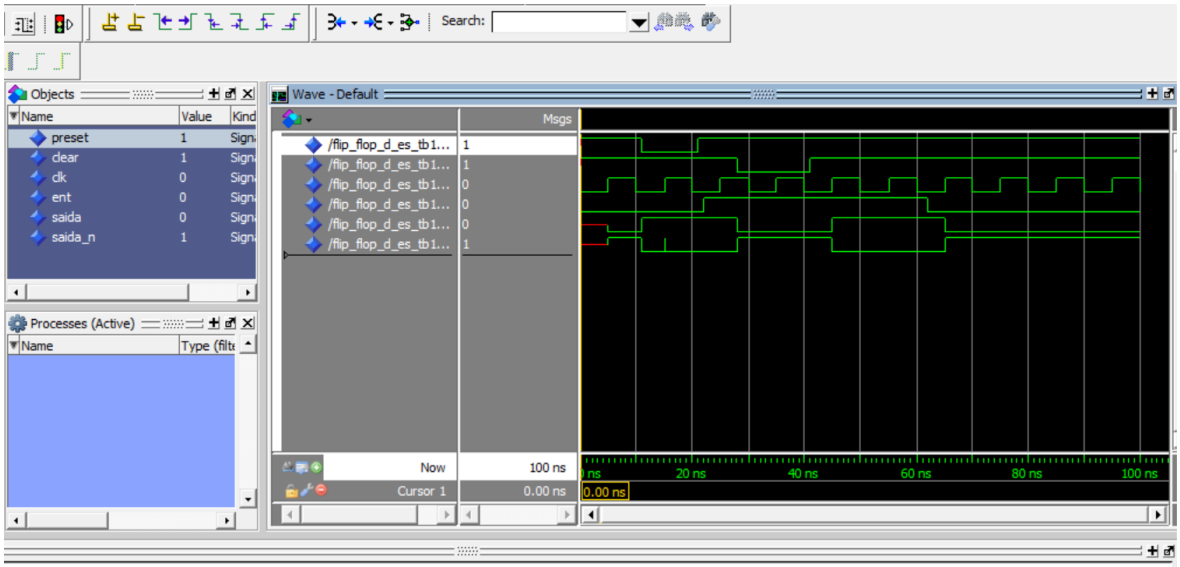
12.2 ESTÍMULOS E LEITURA DOS RESULTADOS

Os tempos do testbench fornecido foram ajustados para que as entradas não mudassem junto das transições do relógio, conforme a orientação dos slides. Os três testes utilizaram as mesmas atribuições:

```
preset <= '1', '0' after 11 ns, '1' after 21 ns;
clear  <= '1', '0' after 28 ns, '1' after 41 ns;
clk    <= not clk after 5 ns;
ent    <= '0', '1' after 22 ns, '0' after 62 ns;
```

Selecionou-se cada teste em Compile test bench e executou-se Tools → Run Simulation Tool → RTL Simulation. Na janela Wave, observaram-se preset, clear, clk, ent, saida e saida_n. Os sinais ent, saida e saida_n correspondem a d, q e q_n. Zoom Full exibiu os 100 ns.

Figura 15 — Simulação de codeonly_arch pelo primeiro testbench.



Fonte: captura de tela da autora (2026).

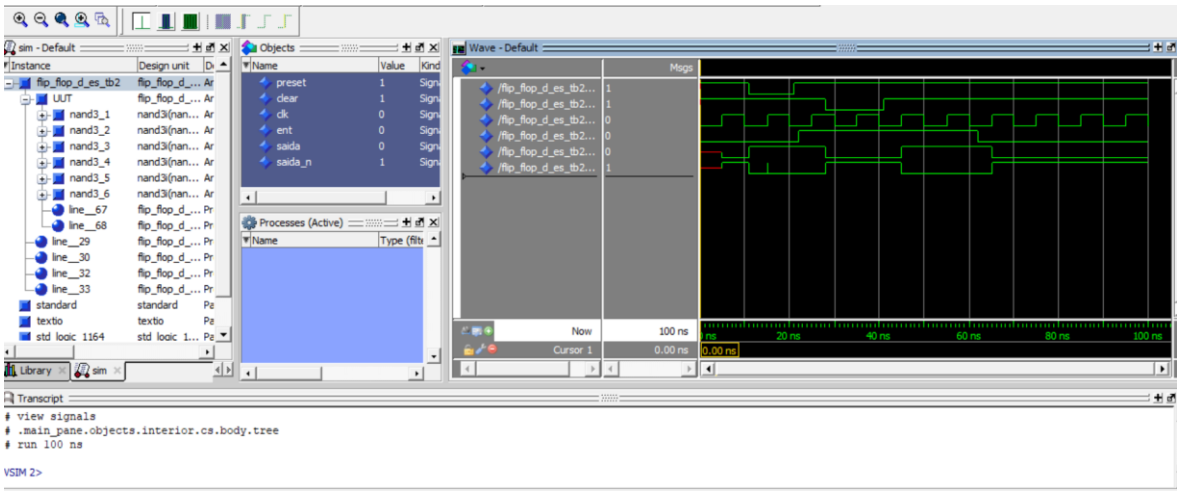
Aos 11 ns, o preset força saida = 1. Aos 28 ns, o clear força saida = 0. Após a liberação do clear aos 41 ns, a saída armazena ent = 1 na borda de subida aos 45 ns. Quando ent muda para 0 aos 62 ns, a saída conserva 1 até a borda aos 65 ns.

Os valores vermelhos no início representam sinais ainda indefinidos. Há também transições estreitas no modelo de portas; a leitura dos estados foi feita após a estabilização de cada evento.

12.3 COMPARAÇÃO DAS VERSÕES HIERÁRQUICAS

Após cada execução, abriu-se View → Structure no ModelSim e expandiu-se UUT na janela sim. O segundo testbench carregou nand3i_arch, cuja hierarquia mostra nand3_1 a nand3_6.

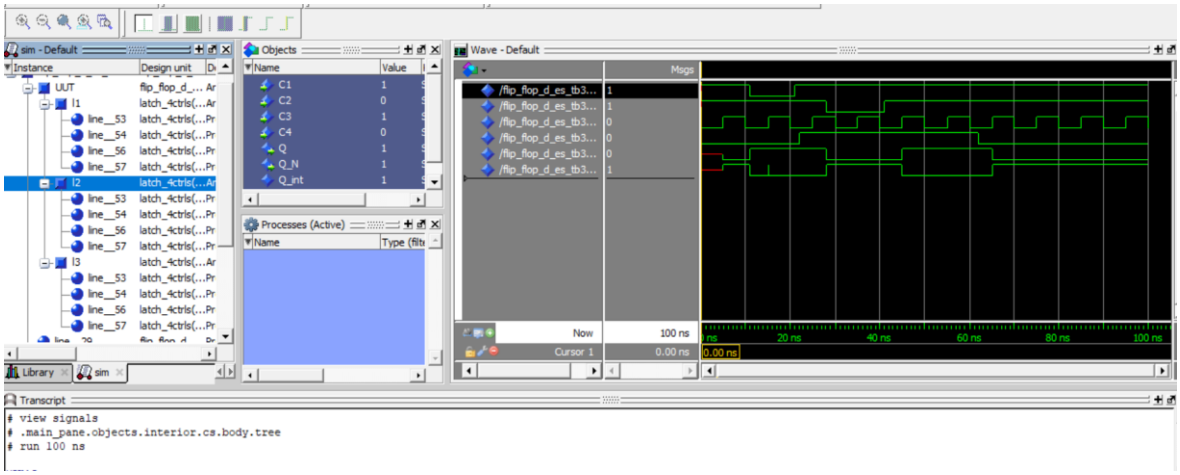
Figura 16 — Seis instâncias NAND e ondas do segundo testbench.



Fonte: captura de tela da autora (2026).

O terceiro testbench carregou latch_arch. Sua hierarquia mostra l1, l2 e l3, formados pelo módulo latch_4ctrls.

Figura 17 — Três instâncias de latch e ondas do terceiro testbench.



Fonte: captura de tela da autora (2026).

As três execuções chegaram a 100 ns e apresentaram o mesmo comportamento externo nos casos observados. A diferença está na organização da descrição: atribuições diretas, instâncias de portas ou instâncias de latches. A comparação foi visual, sem assert de equivalência automática.

REFERÊNCIAS

ALTERA COMMUNITY. Can’t launch the Modelsim Altera Software. [S. l.], [s. d.]. Disponível em: https://community.altera.com/discussions/quartus-prime/cant-launch-the-modelsim-altera-software/179410. Acesso em: 3 out. 2026.

CALAZANS, Ney Laert Vilar. Linguagem de Descrição de Hardware — Unidade 2. UFSC, DEC, Engenharia de Computação. Material fornecido, última alteração em 02/04/2026. Tutorial 1, Projetos 1 a 3. Arquivos VHDL do pacote Tut1_sources, com atualizações indicadas entre 22 e 25/08/2025.

INTEL/ALTERA. Simulating in ModelSim Altera through NativeLink. [S. l.], [s. d.]. Disponível em: https://www.intel.com/content/dam/support/us/en/programmable/kdb/rd10312011-847/nativelink-simulation.pdf. Acesso em: 3 out. 2026.